POLITÉCNICO GRANCOLOMBIANO

Institución Universitaria

Facultad de Ingeniería, Diseño e Innovación

Proyecto de software basado en herramientas de integración continua

Entrega 1 – Semana 3

Repositorio en GitHub y construcción de dos contenedores Docker comunicados entre sí

Módulo: Énfasis Profesional I (Integración Continua)

Integrantes

[Nombre completo del integrante 1]

[Nombre completo del integrante 2]

[Nombre completo del integrante 3]

Tutor: [Nombre del tutor]

Noviembre de 2026

Tabla de contenido

1. Introducción 3
2. Objetivos 3
2.1. Objetivo general 3
2.2. Objetivos específicos 3
3. Marco conceptual 4
3.1. Integración continua 4
3.2. Control de versiones y ambientes 4
3.3. Máquinas virtuales y contenedores 4
4. Descripción del proyecto 5
5. Repositorio en GitHub 6
5.1. Estructura del repositorio 6
5.2. Forma de trabajo 6
6. Construcción de los contenedores 6
6.1. Contenedor 1: aplicación web 6
6.2. Contenedor 2: base de datos 7
6.3. Orquestación con Docker Compose 8
7. Comunicación entre los contenedores 8
8. Ejecución y evidencias 9
8.1. Construir y levantar 9
8.2. Imágenes creadas 9
8.3. Contenedores en ejecución 10
8.4. Los dos contenedores están en la misma red 10
8.5. El contenedor web resuelve el nombre del contenedor de base de datos 10
8.6. La aplicación consulta la base de datos 10
8.7. Prueba de extremo a extremo: escribir por la web y leer en la base 10
8.8. La aplicación en el navegador 11
8.9. Pruebas automatizadas 11
9. Comandos de Docker utilizados 12
10. Problemas encontrados y solución 12
11. Responsabilidades del equipo 12
12. Conclusiones 13
Referencias 13

1. Introducción

El proyecto del módulo consiste en un software que se apoya en herramientas de integración continua. Para esta primera entrega la guía pide dos cosas: que el proyecto ya exista dentro de un repositorio de GitHub y que, usando Docker, se construyan dos contenedores comunicados entre sí.

El software elegido es un **gestor de tareas**: una aplicación web pequeña que permite registrar tareas, consultarlas y marcarlas como completadas. Es un caso sencillo a propósito, porque el objetivo del módulo no es la complejidad de la aplicación sino el proceso que la rodea: versionar el código, empaquetarlo en contenedores y, en las próximas entregas, automatizar su construcción y sus pruebas con Jenkins, Travis CI y Codeship.

El documento explica cómo se organizó el repositorio, cómo se construyó cada contenedor, cómo se comunican y qué evidencias demuestran que la comunicación funciona. Todas las salidas de consola que aparecen son reales: se obtuvieron al ejecutar el proyecto el 10 de noviembre de 2026.

2. Objetivos

2.1. Objetivo general

Crear el proyecto de software en un repositorio de GitHub y desplegarlo en dos contenedores Docker que se comuniquen entre sí, como base para aplicar integración continua en las siguientes entregas.

2.2. Objetivos específicos

- Organizar el código en un repositorio Git con una estructura clara y un historial de cambios trazable.
- Construir una imagen Docker para la aplicación web y otra para la base de datos, cada una con su Dockerfile.
- Comunicar los dos contenedores por una red de Docker, sin depender de direcciones IP fijas.
- Comprobar con evidencias que la aplicación lee y escribe datos en el contenedor de base de datos.
- Dejar pruebas automatizadas mínimas que Jenkins pueda ejecutar en la Entrega 2.

3. Marco conceptual

3.1. Integración continua

Fowler (2006) define la integración continua como una práctica en la que cada miembro del equipo integra su trabajo con frecuencia, por lo general al menos una vez al día, y cada integración se verifica con una construcción y unas pruebas automáticas para detectar los errores lo antes posible. Frente al método tradicional, que según la Lectura fundamental 1 resulta lento, costoso y susceptible de errores humanos, la integración continua acorta las iteraciones y da realimentación constante (Oliveros, 2017a).

La misma lectura enumera las condiciones de un sistema de integración ideal: el código vive en un repositorio, las pruebas se ejecutan con un mínimo de comandos, cualquier interesado puede descargar, compilar y probar la última versión, y todos trabajan sobre la versión más reciente. Este proyecto se diseñó para cumplir esas cuatro condiciones desde la primera entrega.

3.2. Control de versiones y ambientes

Git es un sistema de control de versiones distribuido: cada colaborador tiene una copia completa del repositorio, de modo que si el servidor falla el historial se puede restaurar desde cualquier equipo. Entre los servicios que alojan repositorios Git (GitHub, GitLab, Bitbucket, Coding) se eligió GitHub, que es el que indica la guía y el de mayor comunidad (Oliveros, 2017b).

La Lectura fundamental 2 recomienda al menos tres ambientes (desarrollo, pruebas y producción), cada uno con su propia base de datos y su propio servidor. Los contenedores facilitan justamente eso: el mismo par de imágenes puede levantarse como ambiente de desarrollo en el computador de cada integrante y como ambiente de pruebas en el servidor de integración, con la garantía de que ambos son iguales.

3.3. Máquinas virtuales y contenedores

Una máquina virtual incluye su propio sistema operativo completo sobre un hipervisor, lo que consume más memoria, más disco y más tiempo de configuración. Un contenedor, en cambio, es un paquete ejecutable y liviano que incluye lo necesario para correr la aplicación pero comparte el kernel del equipo que lo aloja. La Lectura fundamental 3 resume sus ventajas en tres características: es ligero, es estándar (corre igual en Linux, Windows o la nube) y es seguro, porque aísla las aplicaciones entre sí y del equipo anfitrión (Oliveros, 2017c).

Los conceptos de Docker que se usan en esta entrega son la **imagen** (la plantilla de solo lectura), el **contenedor** (una imagen en ejecución), el **Dockerfile** (el archivo con las instrucciones para construir una imagen) y la **orquestación**, que la lectura describe como la administración conjunta de varios contenedores con funciones como el descubrimiento de

servicios y el monitoreo de salud. Para orquestar los dos contenedores se usó Docker Compose, una de las herramientas que la lectura menciona.

4. Descripción del proyecto

El gestor de tareas tiene dos componentes, y cada uno corre en su propio contenedor:

Contenedor	Imagen base	Función	Puerto
tareas_web	python:3.12-slim	Aplicación web en Flask, servida con Gunicorn. Ofrece una página y una API REST.	5000 (publicado en el 8085 del equipo)
tareas_db	postgres:16-alpine	Base de datos PostgreSQL que guarda las tareas.	5432 (solo dentro de la red Docker)

Tabla 1. Contenedores del proyecto. Fuente: elaboración propia.

Funciones de la aplicación:

Ruta	Método	Qué hace
/	GET	Página con la lista de tareas y el formulario para agregar.
/tareas	POST	Crea una tarea desde el formulario.
/tareas/<id>/completar	POST	Marca una tarea como completada.
/api/tareas	GET / POST	Lista o crea tareas en formato JSON.
/health	GET	Verifica que el contenedor web puede consultar al de base de datos.

Tabla 2. Rutas de la aplicación. Fuente: elaboración propia.

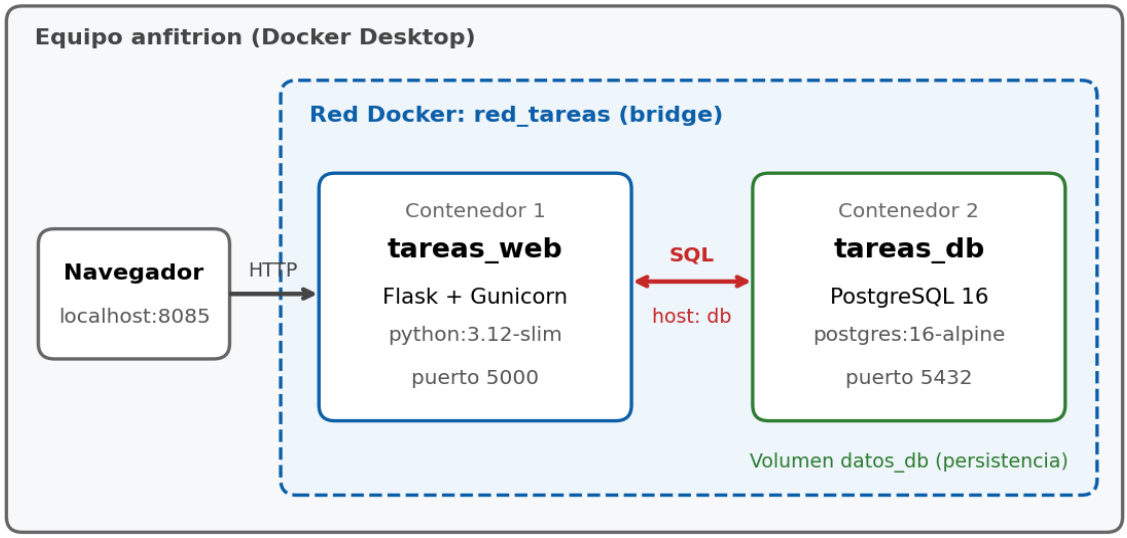


Figura 1. Arquitectura de los dos contenedores. Fuente: elaboración propia.

El navegador solo llega al contenedor web, por el puerto 8085 del equipo. El contenedor de base de datos no publica ningún puerto hacia afuera: únicamente el contenedor web puede alcanzarlo, a través de la red interna `red_tareas`. Esto aplica la característica de seguridad por aislamiento que describe la Lectura fundamental 3.

5. Repositorio en GitHub

El proyecto se versiona con Git en el repositorio **gestor-tareas-ci**. Dirección del repositorio: **[pegar aquí la URL de GitHub]**.

5.1. Estructura del repositorio

```
gestor-tareas-ci/
├── docker-compose.yml     # orquesta los dos contenedores y la red
├── .env.example           # variables de entorno de ejemplo
├── .gitignore
├── README.md              # instrucciones de uso
├── web/                   # contenedor 1: aplicación
│   ├── Dockerfile
│   ├── app.py
│   ├── requirements.txt
│   └── tests/test_app.py
├── db/                    # contenedor 2: base de datos
│   ├── Dockerfile
│   └── init.sql
└── docs/                  # este documento y las evidencias
```

Figura 2. Estructura de carpetas del repositorio. Fuente: elaboración propia.

5.2. Forma de trabajo

- La rama `main` contiene siempre una versión que construye y funciona.
- Cada cambio se trabaja en una rama propia (por ejemplo `feature/completar-tarea`) y entra a `main` mediante un pull request revisado por otro integrante.
- Los mensajes de commit describen qué cambió y por qué, para que el historial sirva como registro de cambios en la Entrega 3.
- Las claves no se suben al repositorio: el archivo `.env` está en `.gitignore` y solo se versiona `.env.example`.

6. Construcción de los contenedores

6.1. Contenedor 1: aplicación web

Archivo `web/Dockerfile`:

```
# Contenedor 1: aplicacion web (Flask)
FROM python:3.12-slim

WORKDIR /app
```

```
# Primero las dependencias: asi Docker reutiliza esta capa si solo cambia el codigo
COPY requirements.txt .
# El "secret" ca_cert es opcional: solo hace falta en equipos donde un antivirus
# o proxy inspecciona el trafico HTTPS (ver README, seccion "Problemas conocidos").
RUN --mount=type=secret,id=ca_cert,required=false \
    if [ -s /run/secrets/ca_cert ]; then export PIP_CERT=/run/secrets/ca_cert; fi && \
    pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

Figura 3. Dockerfile de la aplicación web. Fuente: elaboración propia.

- **FROM**: parte de la imagen oficial de Python en su variante reducida, para que la imagen final sea liviana.
- **COPY requirements.txt y RUN pip install**: se instalan primero las dependencias. Así Docker guarda esa capa en caché y solo la repite cuando cambian las dependencias, no cada vez que cambia el código.
- **EXPOSE 5000**: documenta el puerto en el que escucha la aplicación.
- **CMD**: arranca la aplicación con Gunicorn, un servidor apto para producción.

6.2. Contenedor 2: base de datos

Archivo `db/Dockerfile`:

```
# Contenedor 2: base de datos (PostgreSQL)
FROM postgres:16-alpine

# Todo .sql copiado aqui se ejecuta solo la primera vez que arranca la base
COPY init.sql /docker-entrypoint-initdb.d/01-init.sql

EXPOSE 5432
```

Figura 4. Dockerfile de la base de datos. Fuente: elaboración propia.

La imagen oficial de PostgreSQL ejecuta, la primera vez que arranca, todo archivo `.sql` que encuentre en `/docker-entrypoint-initdb.d/`. Por eso basta copiar `init.sql` para que la tabla y los datos de ejemplo queden creados sin pasos manuales:

```
CREATE TABLE IF NOT EXISTS tareas (
    id          SERIAL PRIMARY KEY,
    titulo      VARCHAR(120) NOT NULL,
    completada  BOOLEAN NOT NULL DEFAULT FALSE,
    creada      TIMESTAMP NOT NULL DEFAULT NOW()
);

INSERT INTO tareas (titulo, completada) VALUES
    ('Crear el repositorio en GitHub', TRUE),
    ('Construir los dos contenedores con Docker', FALSE),
    ('Configurar Jenkins (Entrega 2)', FALSE);
```

Figura 5. Script de inicialización de la base de datos. Fuente: elaboración propia.

6.3. Orquestación con Docker Compose

Archivo `docker-compose.yml`:

```
# Levanta los dos contenedores y la red que los comunica:
#   docker compose up -d --build
services:
  db:
    build: ./db
    container_name: tareas_db
    environment:
      POSTGRES_DB: ${DB_NAME:-tareas}
      POSTGRES_USER: ${DB_USER:-tareas_user}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-cambiar_esta_clave}
    volumes:
      - datos_db:/var/lib/postgresql/data
    networks:
      - red_tareas
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-tareas_user} -d ${DB_NAME:-tareas}"]
      interval: 5s
      timeout: 3s
      retries: 10

  web:
    build: ./web
    container_name: tareas_web
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: ${DB_NAME:-tareas}
      DB_USER: ${DB_USER:-tareas_user}
      DB_PASSWORD: ${DB_PASSWORD:-cambiar_esta_clave}
    ports:
      - "${WEB_PORT:-8085}:5000"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - red_tareas

networks:
  red_tareas:
    driver: bridge

volumes:
  datos_db:
```

Figura 6. Archivo docker-compose.yml. Fuente: elaboración propia.

7. Comunicación entre los contenedores

La comunicación se resuelve con cuatro decisiones del archivo anterior:

1. **Red propia.** Los dos servicios se conectan a `red_tareas`, una red de tipo bridge definida por el proyecto. Solo los contenedores de esa red se ven entre sí.
2. **Descubrimiento por nombre.** Dentro de una red definida por el usuario, Docker incluye un DNS interno que traduce el nombre del servicio a la IP del contenedor. La aplicación se conecta al host `db`, no a una IP: si el contenedor se recrea y cambia de

dirección, la conexión sigue funcionando. Es el descubrimiento de servicios que describe la Lectura fundamental 3.

3. **Configuración por variables de entorno.** El host, el puerto, el usuario y la clave de la base llegan a la aplicación como variables de entorno. El mismo código sirve para desarrollo, pruebas y producción cambiando solo esas variables.
4. **Orden de arranque y salud.** La base de datos tiene un healthcheck con `pg_isready` y el servicio web declara `depends_on` con la condición `service_healthy`: la aplicación no arranca hasta que la base acepta conexiones. Como respaldo, el código reintenta la conexión.

En el código de la aplicación la conexión queda así:

```
DB_CONFIG = {
    "host": os.environ.get("DB_HOST", "db"),
    "port": int(os.environ.get("DB_PORT", "5432")),
    "dbname": os.environ.get("DB_NAME", "tareas"),
    "user": os.environ.get("DB_USER", "tareas_user"),
    "password": os.environ.get("DB_PASSWORD", ""),
}

def conectar(reintentos=10, espera=2):
    for _ in range(reintentos):
        try:
            return psycopg2.connect(connect_timeout=5, **DB_CONFIG)
        except psycopg2.OperationalError as error:
            ultimo_error = error
            time.sleep(espera)
    raise ultimo_error
```

Figura 7. Conexión de la aplicación con el contenedor de base de datos (web/app.py). Fuente: elaboración propia.

8. Ejecución y evidencias

8.1. Construir y levantar

Con Docker Desktop encendido, un solo comando construye las dos imágenes, crea la red y el volumen, y arranca los contenedores:

```
$ docker compose up -d --build
 Network gestor-tareas-ci_red_tareas  Created
 Volume gestor-tareas-ci_datos_db     Created
 Container tareas_db                  Healthy
 Container tareas_web                 Started
```

Figura 8. Construcción y arranque de los contenedores. Fuente: elaboración propia.

8.2. Imágenes creadas

```
$ docker images --filter "reference=gestor-tareas-ci*"
REPOSITORY             TAG       IMAGE ID       SIZE
gestor-tareas-ci-web   latest    bf69a99d91b2   216MB
gestor-tareas-ci-db    latest    eca7e5b8d6c3   419MB
```

Figura 9. Las dos imágenes construidas (docker images). Fuente: elaboración propia.

8.3. Contenedores en ejecución

```
$ docker ps --filter "name=tareas_"
NAMES        IMAGE                  STATUS                    PORTS
tareas_web   gestor-tareas-ci-web   Up 8 seconds              0.0.0.0:8085->5000/tcp, [::]:8085-
>5000/tcp
tareas_db    gestor-tareas-ci-db    Up 14 seconds (healthy)   5432/tcp
```

Figura 10. Los dos contenedores en ejecución (docker ps). Fuente: elaboración propia.

El contenedor de base de datos aparece como healthy y el web publica su puerto 5000 en el 8085 del equipo.

8.4. Los dos contenedores están en la misma red

```
$ docker network inspect gestor-tareas-ci_red_tareas
gestor-tareas-ci_red_tareas (bridge)
  tareas_db -> 172.19.0.2/16
  tareas_web -> 172.19.0.3/16
```

Figura 11. Contenedores conectados a la red red_tareas (salida resumida). Fuente: elaboración propia.

8.5. El contenedor web resuelve el nombre del contenedor de base de datos

```
$ docker exec tareas_web python -c "import socket; print('db resuelve a',
socket.gethostbyname('db'))"
db resuelve a 172.19.0.2
```

Figura 12. Resolución del nombre db desde el contenedor web. Fuente: elaboración propia.

La dirección que devuelve el DNS interno (`172.19.0.2`) es la misma que la red asignó a `tareas_db` en la figura 11.

8.6. La aplicación consulta la base de datos

```
$ curl -s http://localhost:8085/health
{"contenedor_bd":"db","contenedor_web":"93e966689263","estado":"ok","ip_bd":"172.19.0.2","motor":"P
ostgreSQL 16.15 on x86_64-pc-linux-musl"}
```

Figura 13. Respuesta de /health: el contenedor web obtuvo la versión del motor desde el contenedor de base de datos. Fuente: elaboración propia.

8.7. Prueba de extremo a extremo: escribir por la web y leer en la base

Se crea una tarea a través de la API del contenedor web:

```
$ curl -s -X POST http://localhost:8085/api/tareas -H "Content-Type: application/json" -d
'{"titulo":"Probar la comunicacion entre contenedores"}'
{"id":4,"titulo":"Probar la comunicacion entre contenedores"}
```

Figura 14. Creación de una tarea por la API. Fuente: elaboración propia.

Y se consulta directamente dentro del contenedor de base de datos, sin pasar por la aplicación:

```
$ docker exec tareas_db psql -U tareas_user -d tareas -c "SELECT id, titulo, completada FROM tareas
ORDER BY id;"
 id |                 titulo                  | completada
----+-----------------------------------------+------------
  1 | Crear el repositorio en GitHub          | t
  2 | Construir los dos contenedores con Docker | f
  3 | Configurar Jenkins (Entrega 2)          | f
  4 | Probar la comunicacion entre contenedores | f
(4 rows)
```

Figura 15. La tarea 4, creada desde el contenedor web, quedó guardada en el contenedor de base de datos. Fuente: elaboración propia.

Esta es la evidencia principal de la entrega: un dato que entró por un contenedor aparece almacenado en el otro.

8.8. La aplicación en el navegador



Figura 16. Página del gestor de tareas en http://localhost:8085. Fuente: elaboración propia.

La página muestra el identificador del contenedor web que atendió la petición y el motor de base de datos que respondió desde el otro contenedor.

8.9. Pruebas automatizadas

```
$ docker exec tareas_web python -m pytest -q
....                                                                     [100%]
4 passed in 0.10s
```

Figura 17. Ejecución de las pruebas unitarias dentro del contenedor web. Fuente: elaboración propia.

Las cuatro pruebas validan las reglas del título de una tarea y la respuesta de la API ante datos inválidos. Son pocas, pero cumplen lo que pide la Lectura fundamental 1: se ejecutan

con un solo comando y comparan un valor esperado con uno obtenido. Serán el punto de partida del pipeline de Jenkins en la Entrega 2.

9. Comandos de Docker utilizados

Comando	Para qué se usó
docker compose up -d --build	Construir las imágenes y levantar los dos contenedores.
docker images	Listar las imágenes disponibles en el equipo.
docker ps	Listar los contenedores en ejecución.
docker network inspect	Ver qué contenedores están en la red y con qué IP.
docker exec	Ejecutar un comando dentro de un contenedor que ya está corriendo.
docker logs	Revisar la salida de un contenedor.
docker compose down	Detener y eliminar los contenedores (con -v también el volumen).

Tabla 3. Comandos de Docker utilizados. Fuente: elaboración propia.

10. Problemas encontrados y solución

Problema	Causa	Solución
El daemon de Docker no respondía al ejecutar el primer comando.	Docker Desktop no estaba encendido.	Iniciar Docker Desktop y esperar a que el motor esté disponible antes de construir.
La construcción de la imagen web falló con CERTIFICATE_VERIFY_FAILED al instalar las dependencias.	El antivirus del equipo inspecciona el tráfico HTTPS y vuelve a firmar la conexión con un certificado propio que pip, dentro del contenedor, no reconoce.	Se entregó el certificado del antivirus a la construcción como secret de Docker (archivo local docker-compose.override.yml, que no se sube al repositorio). El certificado solo se usa durante pip install y no queda guardado en la imagen. No se desactivó la verificación de certificados.
La aplicación podría arrancar antes de que la base de datos acepte conexiones.	Los dos contenedores inician casi al mismo tiempo y PostgreSQL tarda unos segundos en estar listo.	Healthcheck en la base de datos, depends_on con condición service_healthy y reintentos de conexión en el código.

Tabla 4. Problemas encontrados y solución. Fuente: elaboración propia.

11. Responsabilidades del equipo

Integrante	Responsabilidad en la Entrega 1
[Integrante 1]	Creación del repositorio en GitHub, estructura del proyecto y README.
[Integrante 2]	Aplicación web, su Dockerfile y las pruebas unitarias.

Integrante	Responsabilidad en la Entrega 1
[Integrante 3]	Base de datos, docker-compose, verificación de la comunicación y documento.

Tabla 5. Responsabilidades del equipo. Fuente: elaboración propia.

12. Conclusiones

- El proyecto quedó versionado en Git y empaquetado en dos contenedores que se comunican por una red propia de Docker. La prueba de extremo a extremo lo confirma: una tarea creada por el contenedor web quedó almacenada en el contenedor de base de datos.
- Conectar los contenedores por nombre de servicio y no por IP hace que el despliegue sea repetible en cualquier equipo, que es la condición para automatizarlo.
- Cualquier integrante puede descargar, construir y probar el proyecto con tres comandos, tal como lo plantea la Lectura fundamental 1 para un sistema de integración ideal.
- El error de certificados mostró una ventaja práctica de los contenedores: el problema era del equipo anfitrión y se resolvió sin tocar el código ni dejar rastros en la imagen.
- Lo que sigue es conectar Jenkins al repositorio para que construya las imágenes y ejecute las pruebas en cada cambio (Entrega 2).

Referencias

Docker Inc. (s.f.). Dockerfile reference. Docker Docs. https://docs.docker.com/reference/dockerfile/

Docker Inc. (s.f.). Networking in Compose. Docker Docs. https://docs.docker.com/compose/how-tos/networking/

Fowler, M. (2006). Continuous integration. https://martinfowler.com/articles/continuousIntegration.html

Oliveros, E. (2017a). Fundamentos de la integración continua [Lectura fundamental, Unidad 1, Escenario 1]. Politécnico Grancolombiano.

Oliveros, E. (2017b). Manejo de repositorios [Lectura fundamental, Unidad 1, Escenario 2]. Politécnico Grancolombiano.

Oliveros, E. (2017c). Máquinas virtuales y Dockers para construcción de ambientes [Lectura fundamental, Unidad 2, Escenario 3]. Politécnico Grancolombiano.

Politécnico Grancolombiano. (s.f.). Proyecto de software basado en herramientas de integración continua [Guía del proyecto grupal]. Módulo Énfasis Profesional I (Integración Continua).